PioDeploy Developer Knowledge Base

developerkbr — every page, what we added, and why. Companion to the User Guide (which explains *how to use*; this explains *what exists and the reasoning behind it*).

Generated 2026-07-20 · covers the platform through commit 64ac326

Contents

1. Marketing site (public)

WHAT	WHY WE ADDED IT
Signup wizard (/signup)	The pricing CTA used to dead-end in a contact form; a lead that must wait for a phone call is a lead that shops elsewhere. Four steps (fleet → account → company → review & pay), each validating its own fields; nothing is written until final submit, so abandoning leaves no residue. Password is hashed at submit and the plaintext dropped from component state. Ends in Stripe Checkout (signup_id in metadata) or, with billing off, an invoice-me application.
Approval gate (Admin → Signups)	Payment is machine-verified (webhook flips the row to Paid) but account creation stays a human decision — fraud, typos and sanctioned regions are judgement calls. Approve creates Client + owner User (client-bound Manager) in ONE transaction and sends the welcome mail after it commits; a half-created account is worse than a failed click. The approve button warns loudly when Stripe has NOT confirmed the money.
Team page (client-run)	Owners staff their own tenant without touching the admin area. Binding comes from the session, never the form; only Technician/Viewer are grantable (owners cannot mint peers or admins); removal of self or a fellow owner is refused. Tenancy is enforced on every action, not just the view.
Home page	Animated hero, live-feel dashboard mock, testimonials, comparison table, FAQ with internal/outbound links. The conversion funnel starts here; every section pushes toward Pricing → Request access.
/features page	Was only a home-page anchor; a dedicated page is SEO-indexable, gives every capability its own copy, and ends in the same CTA. Six capability areas + the full 28-item grid.
/pricing	Four headline plan cards (20/50/100/250) instead of seven — one clean row; larger fleets use the calculator + Enterprise quote form so nothing is lost. "Payments secured by Stripe" line answers the card-safety objection at the decision point.
/about	The origin story builds trust; a fragile animated grid was replaced by a plain three-number stat strip after it rendered broken.
/get-started	Accounts are provisioned by staff (no self-service signup) so tenants start correctly; the "What happens next" 3-step section sets expectations after submitting.

SEO layer	sitemap.xml + robots.txt (served by routes so URLs follow APP_URL), canonical/Open Graph/Twitter tags, JSON-LD (Organization, SoftwareApplication, FAQPage), branded 1200×630 og-image. Without these, shared links were bare URLs and Google indexed the app pages.
Logo lockups	Wordmark reads **PioDeploy** (capitalised in the SVGs), sized 50px header / 44px footer; image URLs carry `?v=` because browsers cache SVGs by exact URL.

2. Authentication & security

WHAT	WHY WE ADDED IT
Jetstream login + profile	Standard hardened auth; public registration is intentionally disabled — staff create accounts, so tenancy is always set up correctly.
Two-factor (profile)	Authenticator-app 2FA with recovery codes shipped with Jetstream; we surfaced it because nobody found it.
Require-2FA setting	Settings → Security: off / staff / everyone. Middleware routes unenrolled users to their profile with a banner; profile + 2FA + logout routes stay reachable so enrolment can never dead-lock. Default off so deploying changes nothing.
2FA column on Users	An admin needs fleet-wide visibility of who is enrolled before flipping enforcement on.

3. Dashboard

WHAT	WHY WE ADDED IT
Online / offline tiles	Heartbeat-derived (threshold configurable in Settings). The first thing an MSP checks.
Agents outdated card	Old agents caused real failures (broken winget scan → install loops). Counts machines behind `CURRENT_AGENT_VERSION`, deep-links to the filtered Computers list. Replaced a hardcoded version check that had silently gone stale.
Updates waiting (by package)	One update across sixty machines is one decision, not sixty rows — folded per package.
Browser-policy widget, fleet-by-client, deployments chart	Compliance posture and job health at a glance; each links into its detail page.
Unpaid-subscription banner	Site-wide amber (past due) or red (suspended) bar for staff until the card is fixed — part of the dunning loop. Client-portal users never see it.

4. Clients

WHAT	WHY WE ADDED IT
Client list + CRUD, CSV import/export	The MSP's customer registry; every project, machine and policy hangs off a client.
PDF link (per row)	Downloads a branded compliance report for that client — fleet overview, software/browser policy compliance, 30-day deployment activity, machine list. Built for attaching to the monthly invoice; rendered with dompdf; every query is scoped through the client's projects so cross-tenant leakage is structurally impossible.
Monthly checkbox (per row)	Opt-in per client: when ticked, `reports:client-compliance` emails that client's portal users the PDF on the 1st of each month at 07:00. Default off for every client so enabling the feature changes nothing until you choose. This is the row highlighted in your screenshot.
Client-portal scoping	Client-role users see only their own projects/machines, read-only, with their own dashboard and a self-service compliance-PDF button.

5. Projects & enrolment

WHAT	WHY WE ADDED IT
Projects	The unit of targeting: policies, browser policies and bulk deploys aim at a project; computers belong to one.
Enrolment page (per project)	One PowerShell script per project (GPO / Intune / RMM / manual / uninstall tabs) with the API key baked in; the script self-heals prerequisites (execution policy, VC++ runtime, winget-as-SYSTEM) because real fleets are messy.
Multiple API keys per project	One shared key made rotation an outage: replacing it stopped every enrolled agent at once. Keys are now rows (hash-only, shown once) — one per site/RMM/wave. Creating a key touches nothing; revoking stops only the machines enrolled with that key. The old ⚿ rotate deliberately became "add a key", never "replace".
Project deletion blocked by machines	Machines anchor jobs, history and compliance records; a project must not vanish from under them. Retired machines count too — retirement parks a machine, it does not erase it. Enforced in the service so no caller (or role) can skip it.
Client dunning (fail → remind → suspend → restore)	First failure emails the client instantly from the webhook; a daily command paces reminders (every 3 days, real countdown) and suspends when the operator-set grace closes. Suspension is status + emails, never fleet breakage — cutting machines off over a card decline is a human's call. Payment auto-restores ONLY billing's own suspensions (billing_suspended_at marker) — a manual suspension is a human decision and stays. The past-due clock starts on the first failure and survives Stripe's retries, so repeat failures never spam or reset the countdown.
Recurring billing mirror (per client)	The wizard's checkout creates a real monthly Stripe subscription — Stripe is the system of record and charges on its own. The app only mirrors it (status, amount, renewal) via webhooks: invoice.paid refreshes, payment_failed marks past-due + notifies (fleet keeps working — dunning is Stripe's job, breaking fleets is nobody's), subscription.deleted marks canceled.

	Card management happens on Stripe's hosted Billing Portal, so no card data or cancel logic lives here. Webhook events arrive in any order and repeat, so every handler is idempotent and an unknown subscription is acknowledged, not errored (Stripe retries non-2xx forever).

6. Computers — list

WHAT	WHY WE ADDED IT
Search + client/project/connectivity filters	Finding one machine in a fleet fast (hostname, serial, IP, MAC).
Agent column + "Update available" badge	Stale agents were invisible before; the badge explains itself (self-update on next check-in).
Agent-version filter (outdated / up to date)	URL-bound (`?agentStatus=outdated`) so the dashboard card deep-links straight to the machines that need attention.
Soft-delete + restore	A deleted machine that reports again is revived automatically — agents outlive portal bookkeeping.

7. Computer — detail page

WHAT	WHY WE ADDED IT
Health checks	Derived warnings (never reported, offline > a day, low disk, secure boot off) — MSP triage without digging.
Assignment panel	Shows agent version as "1.0.20 — update available (latest 1.4.0)" against the real latest, not a hardcoded number.
Quick deploy panel	One-off jobs without a policy. The action list adapts to the package type (no Rollback on MSI, no Remove on portable), the button states the outcome before the click ("Upgrade 138 → 141", "Up to date"), and Force is the explicit escape hatch.
One-click "Roll back to X"	Every job snapshots `installed_version_before`; when history knows the last good version, rolling back is one click instead of a version hunt. winget/Chocolatey only — binary installers can't fetch old versions, and we refuse to pretend otherwise.
Installed software (patch table)	Application / Installed / Latest / Update required (Yes-No badge) / Status — the patch-management layout operators know. Opens on **Deployed by PioDeploy** because the first question is "what did WE put here?". **Update now** → on outdated catalogue apps queues the update through the guarded path.
Software status (per policy, in words)	Answers "so why isn't it installed?" when there is no job to point at ("waiting for maintenance window", "ring eligible in 2 hours"…).
Deployment log — collapsed	Every attempt with the agent's own output; collapsed by default with an attempt count because it grows long and is a drill-down, not a summary.
Reinstall agent (header button)	Born from a real incident: a broken self-updater looped and took a machine offline, and the only fix was hands on the keyboard. The button queues a one-shot flag delivered on the next heartbeat; the agent re-downloads the current bundle and swaps itself while keeping its config and identity. Delivered **exactly once** (cleared as it's handed out) so a failed attempt can never loop — a human re-queues it.
Uninstall agent (header button)	Decommissioning shouldn't need a visit either. Same one-shot delivery; the agent deletes its own service and files via a detached scheduled task (a service can't delete itself), logging to Windows temp because every PioDeploy folder is gone when it finishes. The portal record and history stay until deleted deliberately — removal of the agent and removal of the audit trail are different decisions.

Retire vs. Delete permanently	The bin icon retires (soft delete — history kept, auto-revived if the agent reports again). Permanent deletion demands proof the agent is gone: never enrolled, or uninstall delivered and the machine silent since. The proof is self-correcting — any heartbeat clears it, so a lying flag can't outlive a live agent. Guard lives in the service; no caller can bypass it.

8. Device Groups

WHAT	WHY WE ADDED IT
Groups (Fleet → Device Groups)	A hand-picked set of machines that cuts across clients and projects (finance machines, kiosks, pilot rings). A "device tag" is simply membership of a group. Staff-only — grouping is cross-tenant by nature.
Why not Sites/Departments?	They don't exist anywhere in the platform's data model; inventing them only for policies would create orphan concepts. Groups deliver the same targeting power without a platform-wide re-model.

9. Packages

WHAT	WHY WE ADDED IT
Catalogue (~120 seeded + custom)	The approved-software list: winget/Chocolatey ids resolve from their repos; MSI/EXE/MSIX/ZIP carry an installer URL + SHA-256 the agent verifies before executing anything.
Capability matrix	`InstallerType::supports(action)` is the single source of truth for what each type can do (rollback: winget/choco only; uninstall: +MSI). The UI, the queue guard and the policy engine all consult it, so a doomed job can't be created from any door.

10. Deployments & Bulk deploy

WHAT	WHY WE ADDED IT
Deployments list	Live queue + history; repeats of the same task fold into one row with a ×N badge; failed jobs carry a translated failure hint (exit-code → plain English).
Bulk deploy	One package across a whole project (optional ring filter for staged rollouts) with a live target count. Fans out over the same guarded queue, so satisfied machines are skipped and in-flight duplicates collapse — a summary line, not a pile of noise. Rollback is excluded because each machine's previous version differs.

11. Software Policies

WHAT	WHY WE ADDED IT
Desired state (Install / Auto update / Force update / Remove / Block)	Say what the fleet should look like; the agent makes it so and keeps it that way. Modes: Enforce or Audit (report only).
Version modes (Latest / Exact / Minimum / Freeze)	Exact/Freeze can mean a downgrade — the agent runs it as a version-pinned reinstall on package managers; on binary types the policy engine honestly reinstalls current instead of queueing an impossible rollback.
Windows, frequency, rings	Maintenance windows and Pilot→Test→Production rings, because 2 a.m. Saturday and staged rollouts are how real fleets change.
Install-loop guards	Field bug: blind-scan agents (can't run winget as SYSTEM) reported no inventory, so policies reinstalled forever ("Install ×17"). Two fixes: a machine reporting *nothing* from a manager counts a succeeded PioDeploy install as present; and installs are paced to one successful attempt per frequency window regardless.
Batch enforcement	The 5-minute pass asked the DB 15–20 questions per machine per policy (21,305 queries / 10.5 s at 1,000 devices). `PolicyBatchContext` answers them from three set-based queries per policy → 603 queries / 1.7 s. Same tests cover both paths so they can't drift.

9b. Per-tenant packages

WHAT	WHY WE ADDED IT
client_id on packages (null = catalogue)	Customers upload in-house installers and licensed software; those must never be visible to — or deployable for — another tenant. The isolation line is `Package::isUsableFor($project)`, enforced at the ONE funnel every deployment passes through (DeploymentService::queue), so it binds Super Admins too: the guard reads ownership, not the caller's role. Staff may view for support; reuse is impossible by construction.
Tenant packages born private	A tenant's new package is stamped with their client id server-side — not offered as a choice — so a tenant can never publish into the shared catalogue. Pickers everywhere (policy form, deploy forms, update-now) filter to catalogue + own; a smuggled foreign package id fails validation or the funnel guard.

9d. Per-project technician confinement

WHAT	WHY WE ADDED IT
project_user pivot; none = roam	Assigning is how an owner RESTRICTS, not grants: zero rows keeps today's whole-tenant access (small teams need no ceremony); rows confine the user to those projects everywhere — policies (Project/Computer view) block direct URLs, repo/list filters hide rows, deploy targets and template pickers shrink. Owners are never confinable (Team page refuses): restricting the person who does the restricting is a lockout waiting to happen.

9c. Software licenses

WHAT	WHY WE ADDED IT
Per-client license register	Paid keys, seats, expiry — the part of an MSP's estate that lives in spreadsheets. Keys are Crypt-encrypted at rest with only a last-4 fingerprint stored readably; `revealKeyFor()` is the single door to plaintext and refuses everyone outside the owning tenant — staff included. Support can see a license exists; the value stays the client's secret.
Seat assignments	A seat pinned to a computer makes usage honest (3/5 used) and over-allocation impossible — the service refuses past the seat count, idempotently re-assigning the same machine costs nothing, and cross-client assignment is refused by ownership (role-blind, same rule as private packages).

11b. Policy templates

WHAT	WHY WE ADDED IT
One-click starter kits	Onboarding a customer meant creating ~15 policies by hand. A template applies a whole kit at once; items are keyed by **winget id** (the portable identity), so a kit works on a fresh install and even creates missing catalogue packages on apply. Applying is idempotent — existing project+package+action policies are skipped, never duplicated.
Staff-only authoring	Templates are global. A tenant-made template would leak that tenant's software choices to every other customer — so tenants can apply, only staff can create/delete. Built-ins are product-defined and refreshed by the seeder on every deploy; deleting a template never touches policies it created (they belong to their projects).

12. Browser Policies

WHAT	WHY WE ADDED IT
Windows Security types (win_*)	The OS is just another registry surface: Browser::Windows pseudo-target, gated BEFORE the per-browser matches so both families stay exhaustive and can never leak into each other (a Windows type answers only for the OS; browser "all" excludes it). Reuses everything: scoping, manifest rollback, compliance reporting. Agent change was ONE line ("windows" is always installed) — the enforcer was already surface-agnostic. Each type documents its permissive value so enable/delete restores Windows defaults.

WHAT	WHY WE ADDED IT
Catalogue (36 policies, 19 categories)	GPO-grade browser lockdown without a domain: passwords, private browsing, autofill, sync, downloads, cookies, permissions, printing, lockdown, AI assistants (Gemini/Copilot/Leo), Edge extras, WebUSB/BT/Serial. Each policy is one enum case with a per-browser registry/policies.json map — adding one is a single case, nothing else changes. Items the platform can't honestly enforce (kiosk mode, right-click, per-user HKCU) were rejected rather than faked.
Value-typed policies	Forced homepage / new-tab URL, block-all-extensions, force-install extensions by ID (strictly validated 32-char Web Store ids). Needed agent 1.4.0's `registry_sz` / `registry_list` operation kinds; list entries are tracked individually so a shrinking list rolls back exactly the stale numbers.
Assignment scopes + inheritance	All machines / Client / Project / Device group / Single computer. Same-type overlaps resolve by specificity — **Computer › Group › Project › Client › All** — so "block incognito company-wide, allow for the Developers group" just works. Exclusions still override everything. Existing project policies migrated byte-for-byte.
Templates (7 built-in + custom)	High Security, Standard Office, Developer, Kiosk, Finance, Healthcare, Education — code-defined so they need no seeding and can only reference real catalogue types. Apply skips types the target already has; capture any project as a custom template.
Compliance dashboard	"Are we protected, and where not?" — fleet %, per-policy bars, an only-problems filter, and a failing-machines list carrying the agent's own error detail. Tenant-lensed: shared policies count a client's own machines only.
Import / export	Versioned JSON that round-trips values and browser scopes; import becomes a custom template, unknown types are skipped and counted, never silently lost.

13. Reports

WHAT	WHY WE ADDED IT
Compliance / Deployments / Computers (CSV)	The exportable views for evidence and spreadsheets.
Client compliance PDF	See Clients (§4) — the on-demand and monthly branded report.

14. Billing — customer side

WHAT	WHY WE ADDED IT
Plans + calculator + enterprise quotes	Per-endpoint pricing; `PricingService` is the single authoritative engine (the JS calculator only mirrors it).
Trial with card verification	14 days via Stripe SetupIntent — the card never touches our server; prepaid cards are rejected to keep trial abuse down.
Subscription page	Swap/cancel/resume/pause; status is always *derived* from Stripe's local mirror, never guessed.
Invoices	Downloads redirect to Stripe's hosted PDF — no dompdf here, Stripe's document is the authoritative one.
Dunning	Stripe retries fire a failure email each attempt; after Stripe gives up, `billing:dunning-reminders` nudges every 3 days; the site-wide banner keeps it visible; a successful payment resets everything.

15. Billing — admin side

WHAT	WHY WE ADDED IT
Billing overview	MRR/ARR/churn/LTV + payment log + CSV export.
Webhooks monitor	Every Stripe event with verify/idempotency status and a Retry button — webhook debugging without SSH.
Coupons + Affiliates	Growth levers: Stripe-backed coupons with local validation; referral links, commission accrual on paid invoices, payout requests (race-safe via row locks).
Billing settings	Stripe keys entered in the portal (encrypted, write-only fields), webhook secret, tax toggle — no .env editing over SSH.

16. Administration

WHAT	WHY WE ADDED IT
Users / Roles matrix	Spatie permissions behind a tick-box matrix; Client-role accounts are hard-bound to a client.
Settings	Operational knobs (heartbeat threshold, retries, backoff, retention, 2FA requirement) — editable live, no redeploy.
Email (SMTP)	Provider presets + "Send test email", because saving proves nothing and sending does.
Content editor	Marketing copy (hero, CTA, taglines) editable in-portal with reset-to-default.
Activity log	Spatie activity log across sensitive actions; retention pruned nightly.
Impersonation	Support tool with a loud banner and a one-click leave.

17. The agent (.NET 8, v1.4.3)

WHAT	WHY WE ADDED IT
Silent SYSTEM service	Registers, heartbeats, reports inventory, claims jobs, applies browser policies — no user ever sees a window.
Checksum gate	A downloaded binary that doesn't match its catalogue SHA-256 is never executed.
Browser-policy manifest	Everything the agent wrote is recorded locally; anything the server no longer wants is rolled back on the next run — deleting a policy undoes it automatically.
Self-update	The heartbeat advertises the latest version; agents upgrade themselves. The server must have the new bundle *published* or the fleet stays put — the dashboard card shows the spread.
Self-update via Scheduled Task (1.4.2)	The original helper launch used UseShellExecute, which silently does nothing in a service's session 0 — agents looped downloading forever and one ended up offline. A one-shot Scheduled Task is owned by Windows, not by the dying service; the helper now logs before acting so a failed swap is visible, and it never overwrites the machine's `appsettings.json` (the bundle's copy is a placeholder template — copying it disconnected the agent).
winget --scope machine (1.4.1)	winget under SYSTEM defaults to per-user, so installs "succeeded" into the invisible SYSTEM profile. Machine scope makes them real; packages with no machine-wide installer now fail loudly with a hint instead of vanishing.
Remote reinstall / uninstall (1.4.3)	One-shot commands riding the heartbeat: reinstall re-runs the swap regardless of version (the remote fix for a broken install that still checks in); uninstall removes service and files via the same detached-task pattern. Server clears each flag as it is delivered, so neither can ever loop.

18. Under the hood

WHAT	WHY WE ADDED IT
Guarded job queue	`queueIfNeeded` refuses unsupported action/type combos, collapses in-flight duplicates, and skips satisfied machines — every door (quick deploy, bulk, policies, API) goes through it.
Scheduler	`policies:enforce` 5-min, offline check 15-min, drift digest + log prune nightly, trial reminders 09:00, dunning 09:30, client reports monthly. One crontab line runs it all.
perf:bench	Seeds a synthetic fleet (bench-named DB only) and times the hot paths with query counts — so performance claims are measured, not guessed. See docs/PERFORMANCE.md.
Test discipline	~720 PHP + 73 agent tests; every feature phase shipped with tests and the guide/PDF updated in the same commit.

> **Reading this next to the code:** most rows map 1-to-1 to a service class (`app/Services`), a Livewire component (`app/Livewire`) and a feature test (`tests/Feature`) named after the same concept. The commit history tells the same story in order — `git log --oneline` .